

09:00-09:50 (쉬는 시간 11:30-12:00)

PR Target과 Fixture

PR · Commit · pr_target.json

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

data/day4_github/pr_fixture.json
src/course_services/github_service.py

확인할 결과

pr_target.json

4일차 전체 시간표

3시간 블록(30분 휴식) · 2시간 블록(20분 휴식) · 점심 14:00-15:00 · 3시간 블록(30분 휴식)

시간	차시	배우는 내용	진행	확인할 결과
09:00-09:50	1차시	PR Target과 Fixture	강의 12 시연 10 Codex 제작 23 확인 5분	pr_target.json
09:50-10:40	2차시	GitHub 인증과 최소 권한	강의 12 시연 10 Codex 제작 23 확인 5분	.env.sample
10:40-11:30	3차시	REST 상태 코드와 복구	강의 12 시연 10 Codex 제작 23 확인 5분	github_read_result.json
12:00-12:50	4차시	LangGraph State와 분기	강의 12 시연 10 Codex 제작 23 확인 5분	compiled_review_graph
12:50-13:40	5차시	Checkpoint와 Idempotency	강의 12 시연 10 Codex 제작 23 확인 5분	checkpoint_and_audit.json
15:00-15:50	6차시	Human Approval과 Interrupt	강의 12 시연 10 Codex 제작 23 확인 5분	review_decision.json
15:50-16:40	7차시	Dry-run Comment Payload	강의 12 시연 10 Codex 제작 23 확인 5분	review_comment_plan.json
16:40-17:30	8차시	Sandbox GitHub 게시	강의 12 시연 10 Codex 제작 23 확인 5분	day4_audit_record.json

11:30-12:00 쉬는 시간 · 13:40-14:00 쉬는 시간 · 14:00-15:00 점심시간 · 17:30-18:00 쉬는 시간

문제와 필요성

대상 commit이 바뀌면 이전 line mapping과 review comment가 stale 해진다.

repository·PR number·head SHA·diff를 fixture에서 읽는다.

이번 차시의 확인 결과: pr_target.json

차시 목표

- 01 — repository·PR number·head SHA·diff를 fixture에서 읽는다.
- 02 — 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다. 왜 위험한지 설명한다.
- 03 — pr_target.json에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
PR	변경을 검토·병합하는 요청
Commit	특정 시점의 변경 묶음
Head SHA	PR 최신 commit 식별자
Review comment	diff line에 붙는 의견

입력·처리·결과·검증

INPUT

data/day4_github/pr_fixture.json

OUTPUT

pr_target.json

PROCESS

fixture load → target validate → head SHA 고정

VERIFY

synthetic fixture가 PullRequestTarget으로 validate된다.
필드 누락이나 workspace 밖 fixture는 구조화 오류다.

5단계 실행 흐름

01

fixture load

02

target validate

03

head SHA 고정

04

diff load

05

read-only result

마지막 판단은 pr_target.json에서 사람이 확인합니다.

성공·실패 계약

정상	——	synthetic fixture가 PullRequestTarget으로 validate된다.
경계	——	필드 누락이나 workspace 밖 fixture는 구조화 오류다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·pr_target.json

저장소 파일 지도

입력·fixture

data/day4_github/pr_fixture.json

구현 코드

src/course_services/github_service.py

검증·test

data/day3_review_cases/unsafe_pr.diff

따라하기 notebook

materials/day4/day4_service_lab.ipynb

개념과 코드 연결

대상 commit이 바뀌면 이전 line mapping과 review comment가 stale해진다.

쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

이 차시에서
버릴 오해

화면에 보이는 PR
제목만 믿고
repo-number-SHA를
확인하지 않는다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
PR	변경을 검토·병합하는 요청	결정론 test로 먼저 확인
Commit	특정 시점의 변경 묶음	adapter 경계를 확인
Head SHA	PR 최신 commit 식별자	비용·지연·권한을 확인
Review comment	diff line에 붙는 의견	사람 판단과 운영 기록을 확인

대표 실패

실패

화면에 보이는 PR 제목만 믿고
repo·number·SHA를 확인하지
않는다.

사용자 영향

대상 commit이 바뀌면 이전 line mapping과
review comment가 stale해진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: 필드 누락이나 workspace 밖 fixture는 구조화 오류다.
- 02 — 중단: 화면에 보이는 PR 제목만 믿고 repo·number·SHA를 확인하지 않는다.
- 03 — 복구: 쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.
- 04 — 증거: pr_target.json과 test 결과를 함께 남긴다.

Codex 검증 루프

fixture loader에 필수
field-workspace path test를
추가한다.

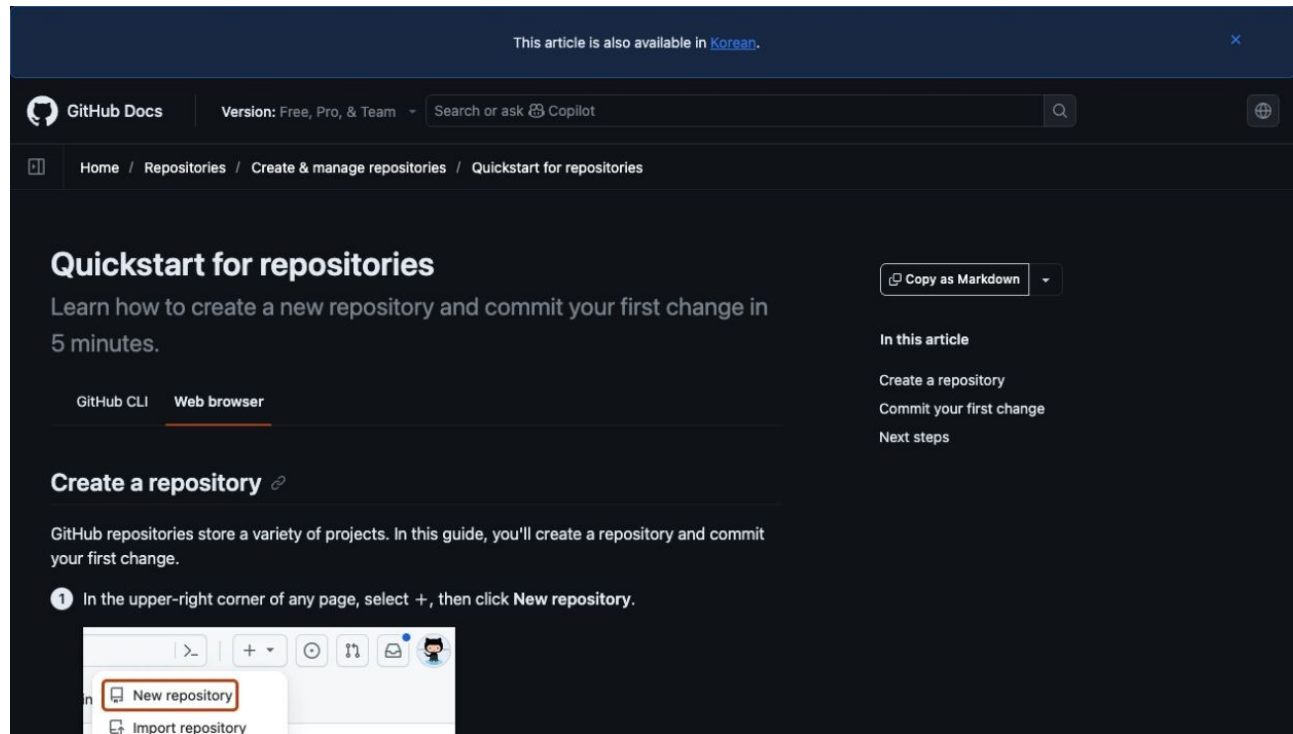
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex 작업 명세

목표

fixture loader에 필수 field·workspace path test를 추가한다.

허용 경로

src/course_services/github_service.py
data/day3_review_cases/unsafe_pr.diff

완료 조건

synthetic fixture가 PullRequestTarget으로 validate된다.
필드 누락이나 workspace 밖 fixture는 구조화 오류다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | synthetic fixture가 PullRequestTarget으로 validate된다. |
| 03 | Boundary test | —— | 필드 누락이나 workspace 밖 fixture는 구조화 오류다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사 정상 경로 시연

- 01 — 열기: `data/day4_github/pr_fixture.json`
- 02 — 구현 확인: `src/course_services/github_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k github`
- 04 — 성공 신호: `synthetic fixture`가 `PullRequestTarget`으로 `validate`된다.

실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k github
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

pr_target.json

사람이 확인할 세 줄

1. synthetic fixture가 PullRequestTarget으로 validate된다.
2. 필드 누락이나 workspace 밖 fixture는 구조화 오류다.
3. external_write=false

강사 실패·복구 시연

재현

화면에 보이는 PR 제목만 믿고
repo-number-SHA를 확인하지 않는다.

복구

쓰기 전 target 세 필드를 payload와
승인 화면에 다시 표시한다.

확인: 필드 누락이나 workspace 밖 fixture는 구조화 오류다.

Codex 기반 소프트웨어 제작

열 파일

src/course_services/github_service.py
data/day3_review_cases/unsafe_pr.diff

작업 범위

Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준

synthetic fixture가 PullRequestTarget으로 validate된다.
pr_target.json 저장

STEP 1 · 작업 명세 입력

fixture loader에 필수 field·workspace path test를 추가한다.

허용 파일: src/course_services/github_service.py ·
data/day3_review_cases/unsafe_pr.diff

결과 파일: pr_target.json

STEP 2 · Patch 생성과 Focused Test

```
python -m pytest -q tests/test_course_services.py -k github
```

정상 Case: synthetic fixture가 PullRequestTarget으로 validate된다.

결과 파일: pr_target.json

STEP 3 · Diff 리뷰와 경계 Case

필드 누락이나 workspace 밖 fixture는 구조화 오류다.
복구: 쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 Case Test

NORMAL

synthetic fixture가 PullRequestTarget으로 validate된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

필드 누락이나 workspace 밖 fixture는 구조화 오류다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

사내 repo PR을 읽어 검토 대기열을 만드는 read-only bot

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — pr_target.json을 운영 검토 자료로 사용

포트폴리오 적용

사내 repo PR을 읽어 검토 대기열을 만드는 read-only bot

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — pr_target.json과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 synthetic fixture가 PullRequestTarget으로 validate된다.로 정의하는가

복구

쓰기 전 target 세 필드를 payload와 승인 화면에 다시 표시한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

pr_target.json에서 어떤 status_error_code를 보는가

1차시 완료 확인

- 01 — 설명: PR Target과 Fixture이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k github`
- 03 — 증거: `pr_target.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: GitHub 인증과 최소 권한

DAY 4 · 2차시

09:50-10:40 (쉬는 시간 11:30-12:00)

GitHub 인증과 최소 권한

PAT · Fine-grained · .env.sample

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

.env.sample
.gitignore

확인할 결과

.env.sample

2차시 입력과 결과

입력	————	pr_target.json
이번 차시	————	GitHub 인증과 최소 권한
결과	————	.env.sample
다음 연결	————	REST 상태 코드와 복구

문제와 필요성

교육용 token 하나가 실수로 commit되면 저장소 전체 위협으로 번진다.

fine-grained token과 최소 repository permission을 설명한다.

이번 차시의 확인 결과: .env.sample

차시 목표

- 01 — fine-grained token과 최소 repository permission을 설명한다.
- 02 — token 값을 notebook output이나 screenshot에 노출한다. 왜 위험한지 설명한다.
- 03 — .env.sample에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
PAT	개인 API 접근 토큰
Fine-grained	대상과 권한을 좁힌 token
Secret	소스·로그에 남기면 안 되는 값
Least privilege	필요한 최소 권한만 부여

입력·처리·결과·검증

INPUT

.env.sample

PROCESS

권한 목록 → sandbox repo → .env local

OUTPUT

.env.sample

VERIFY

.env는 ignore되고 .env.sample만 추적된다.
token이 없으면 401을 재시도하지 않고 실행을
중단한다.

5단계 실행 흐름

01

권한 목록

02

sandbox repo

03

.env local

04

.gitignore

05

완료·폐기

마지막 판단은 .env.sample에서 사람이 확인합니다.

성공·실패 계약

정상	——	.env는 ignore되고 .env.sample만 추적된다.
경계	——	token이 없으면 401을 재시도하지 않고 실행을 중단한다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·env.sample

저장소 파일 지도

입력·fixture

`.env.sample`

구현 코드

`.gitignore`

검증·test

`AGENTS.md`

따라하기 notebook

`data/day4_github/pr_fixture.json`

개념과 코드 연결

교육용 token 하나가 실수로 commit되면 저장소 전체 위험으로 번진다.

존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.

이 차시에서
버릴 오해

token 값을 notebook
output이나 screenshot
에 노출한다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
PAT	개인 API 접근 토큰	결정론 test로 먼저 확인
Fine-grained	대상과 권한을 좁힌 token	adapter 경계를 확인
Secret	소스·로그에 남기면 안 되는 값	비용·지연·권한을 확인
Least privilege	필요한 최소 권한만 부여	사람 판단과 운영 기록을 확인

대표 실패

실패

token 값을 notebook output이나
screenshot에 노출한다.

사용자 영향

교육용 token 하나가 실수로 commit되면
저장소 전체 위협으로 번진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: token이 없으면 401을 재시도하지 않고 실행을 중단한다.
- 02 — 중단: token 값을 notebook output이나 screenshot에 노출한다.
- 03 — 복구: 존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.
- 04 — 증거: .env.sample과 test 결과를 함께 남긴다.

Codex 검증 루프

secret을 읽거나 출력하지 않는
범위에서 config validation만
구현한다.

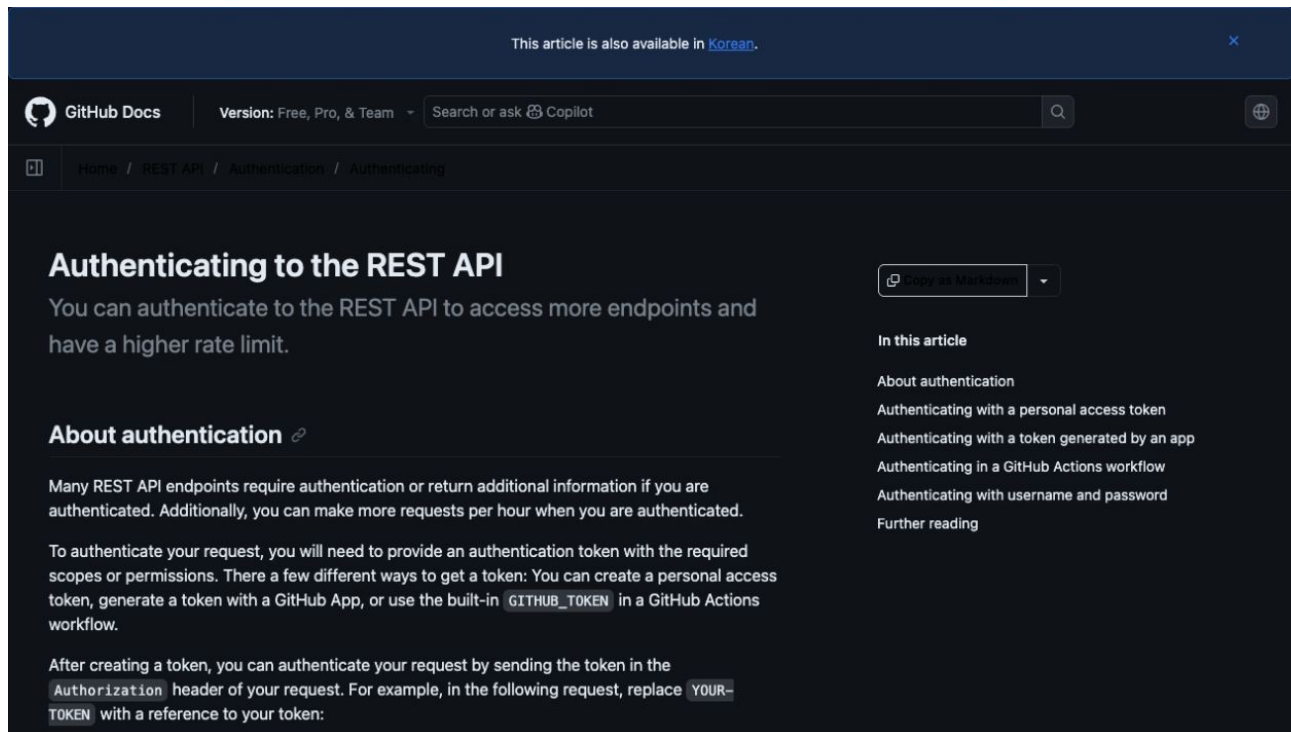
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사랍 merge



Codex 작업 명세

목표

secret을 읽거나 출력하지 않는 범위에서 config validation만 구현한다.

허용 경로

.gitignore
AGENTS.md

완료 조건

.env는 ignore되고 .env.sample만 추적된다.
token이 없으면 401을 재시도하지 않고 실행을 중단한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — .env는 ignore되고 .env.sample만 추적된다.
- 03 **Boundary test** — token이 없으면 401을 재시도하지 않고 실행을 중단한다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사 정상 경로 시연

- 01 — 열기: `.env.sample`
- 02 — 구현 확인: `.gitignore`
- 03 — 실행: `git check-ignore -v .env`
- 04 — 성공 신호: `.env`는 ignore되고 `.env.sample`만 추적된다.

실행 명령

```
$ git check-ignore -v .env
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

.env.sample

사람이 확인할 세 줄

1. .env는 ignore되고 .env.sample만 추적된다.
2. token이 없으면 401을 재시도하지 않고 실행을 중단한다.
3. external_write=false

강사 실패·복구 시연

재현

token 값을 notebook output이나 screenshot에 노출한다.

복구

존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.

확인: token이 없으면 401을 재시도하지 않고 실행을 중단한다.

Codex 기반 소프트웨어 제작

열 파일 .gitignore
 AGENTS.md

작업 범위 Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 .env는 ignore되고 .env.sample만 추적된다.
 .env.sample 저장

STEP 1 · 작업 명세 입력

secret을 읽거나 출력하지 않는 범위에서 config validation만 구현한다.
허용 파일: .gitignore · AGENTS.md

결과 파일: .env.sample

STEP 2 · Patch 생성과 Focused Test

```
git check-ignore -v .env
```

정상 Case: .env는 ignore되고 .env.sample만 추적된다.

결과 파일: .env.sample

STEP 3 · Diff 리뷰와 경계 Case

token이 없으면 401을 재시도하지 않고 실행을 중단한다.
복구: 존재 여부만 bool로 확인하고 노출된 token은 즉시 폐기·재발급한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 Case Test

NORMAL

.env는 ignore되고 .env.sample만 추적된다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

token이 없으면 401을 재시도하지 않고 실행을 중단한다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

교육에서는 mock, 운영에서는 App token·short-lived credential 고려

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — .env.sample을 운영 검토 자료로 사용

포트폴리오 적용

교육에서는 mock, 운영에서는 App token-short-lived credential 고려

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — .env.sample과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 .env는 ignore되고 .env.sample만 추적된다.
로 정의하는가

복구

존재 여부만 bool로 확인하고 노출된 token은 즉시
폐기·재발급한다.

안전

어디에서 사람 승인과 external_write=false를
확인하는가

관측

.env.sample에서 어떤 status_error_code를 보는가

2차시 완료 확인

- 01 — 설명: GitHub 인증과 최소 권한이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `git check-ignore -v .env`
- 03 — 증거: `.env.sample`과 정상·실패 test 결과가 남아 있다.

다음 차시: REST 상태 코드와 복구

DAY 4 · 3차시

10:40-11:30 (쉬는 시간 11:30-12:00)

REST 상태 코드와 복구

REST · Pagination · github_read_result.json

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/github_service.py
materials/day4/day4_service_lab.ipynb

확인할 결과

github_read_result.json

3차시 입력과 결과

입력	——	.env.sample
이번 차시	——	REST 상태 코드와 복구
결과	——	github_read_result.json
다음 연결	——	LangGraph State와 분기

문제와 필요성

모든 오류를 같은 `retry`로 처리하면 인증 문제와 `rate limit`이 장애를 키운다.

read-only PR 조회와 pagination·rate limit 계약을 만든다.

이번 차시의 확인 결과: `github_read_result.json`

차시 목표

- 01 — read-only PR 조회와 pagination·rate limit 계약을 만든다.
- 02 — 401·403·404·422·429를 모두 세 번 retry한다. 왜 위험한지 설명한다.
- 03 — github_read_result.json에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
REST	HTTP resource 단위 API 방식
Pagination	여러 페이지로 결과를 나눔
Rate limit	시간당 요청 제한
Retry-After	다시 요청 가능한 시각 힌트

입력·처리·결과·검증

INPUT

src/course_services/github_service.py

OUTPUT

github_read_result.json

PROCESS

GET request → status classify → pagination

VERIFY

200 응답은 target·diff·rate metadata를 반환한다.
429만 Retry-After와 최대 횟수 안에서 재시도한다.

5단계 실행 흐름

01

GET request

02

status classify

03

pagination

04

bounded backoff

05

structured result

마지막 판단은 `github_read_result.json`에서 사람이 확인합니다.

성공·실패 계약

정상	——	200 응답은 target·diff·rate metadata를 반환한다.
경계	——	429만 Retry-After와 최대 횟수 안에서 재시도한다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·github_read_result.json

저장소 파일 지도

입력·fixture

src/course_services/github_service.py

구현 코드

materials/day4/day4_service_lab.ipynb

검증·test

AGENTS.md

따라하기 notebook

output/day4-github/read-result.json

개념과 코드 연결

모든 오류를 같은 retry로 처리하면 인증 문제와 rate limit
이 장애를 키운다.

재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.

이 차시에서
버릴 오해

401·403·404·422·429
를 모두 세 번 retry
한다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
REST	HTTP resource 단위 API 방식	결정론 test로 먼저 확인
Pagination	여러 페이지로 결과를 나눔	adapter 경계를 확인
Rate limit	시간당 요청 제한	비용·지연·권한을 확인
Retry-After	다시 요청 가능한 시각 힌트	사람 판단과 운영 기록을 확인

대표 실패

실패

401·403·404·422·429를 모두 세 번 retry한다.

사용자 영향

모든 오류를 같은 retry로 처리하면 인증 문제와 rate limit이 장애를 키운다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: 429만 Retry-After와 최대 횟수 안에서 재시도한다.
- 02 — 중단: 401·403·404·422·429를 모두 세 번 retry한다.
- 03 — 복구: 재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.
- 04 — 증거: github_read_result.json과 test 결과를 함께 남긴다.

Codex 검증 루프

fake HTTP response로 각 status code의 결정론 test를 작성한다.

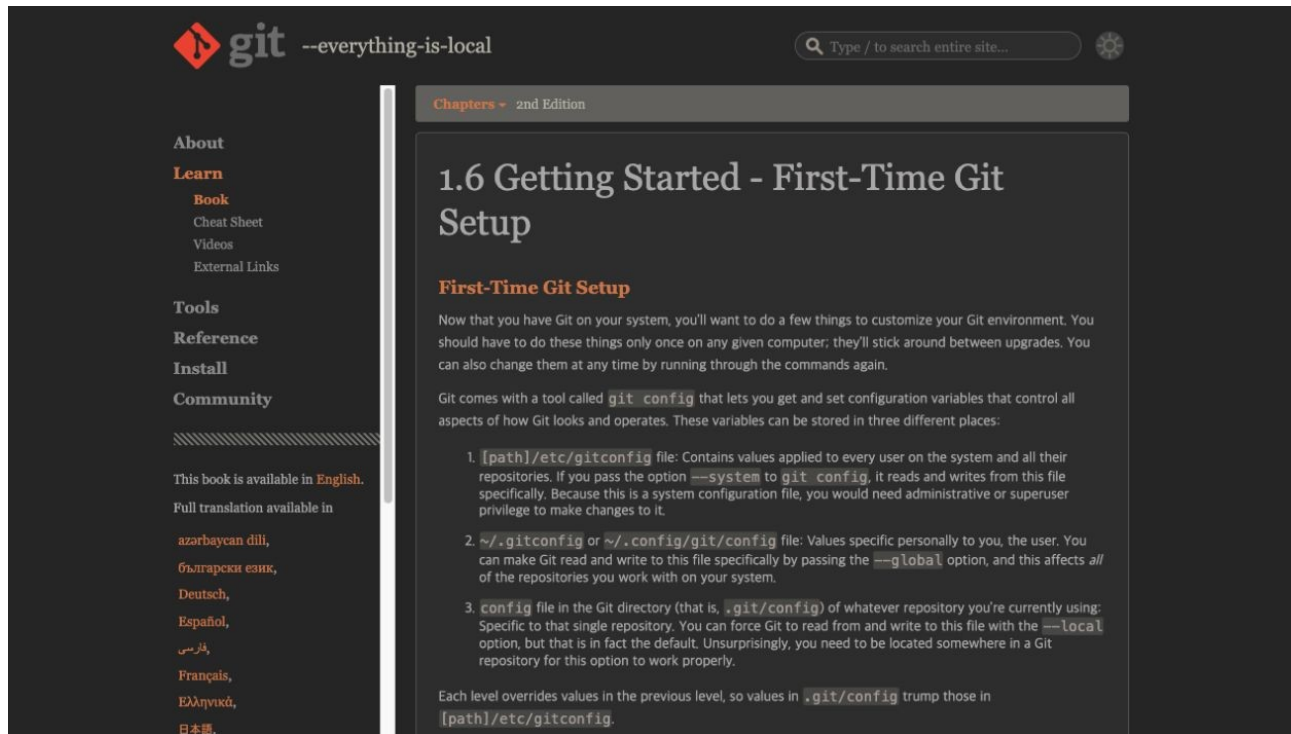
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex 작업 명세

목표

fake HTTP response로 각 status code의 결정론 test를 작성한다.

허용 경로

materials/day4/day4_service_lab.ipynb
AGENTS.md

완료 조건

200 응답은 target-diff-rate metadata를 반환한다.
429만 Retry-After와 최대 횟수 안에서 재시도한다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | 200 응답은 target·diff·rate metadata를 반환한다. |
| 03 | Boundary test | —— | 429만 Retry-After와 최대 횟수 안에서 재시도한다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사 정상 경로 시연

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `materials/day4/day4_service_lab.ipynb`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: 200 응답은 `target·diff·rate metadata`를 반환한다.

실행 명령

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

github_read_result.json

사람이 확인할 세 줄

1. 200 응답은 target·diff·rate metadata를 반환한다.
2. 429만 Retry-After와 최대 횟수 안에서 재시도한다.
3. external_write=false

강사 실패·복구 시연

재현

401·403·404·422·429를 모두 세 번 retry한다.

복구

재인증·권한 확인·target 확인·diff 갱신·
대기 경로를 error code별로 나눈다.

확인: 429만 Retry-After와 최대 횟수 안에서 재시도한다.

Codex 기반 소프트웨어 제작

열 파일	materials/day4/day4_service_lab.ipynb AGENTS.md
작업 범위	Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	200 응답은 target·diff·rate metadata를 반환한다. github_read_result.json 저장

STEP 1 · 작업 명세 입력

fake HTTP response로 각 status code의 결정론 test를 작성한다.
허용 파일: materials/day4/day4_service_lab.ipynb · AGENTS.md

결과 파일: github_read_result.json

STEP 2 · Patch 생성과 Focused Test

```
python -m pytest -q tests/test_course_services.py
```

정상 Case: 200 응답은 target·diff·rate metadata를 반환한다.

결과 파일: github_read_result.json

STEP 3 · Diff 리뷰와 경계 Case

429만 Retry-After와 최대 횟수 안에서 재시도한다.

복구: 재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 Case Test

NORMAL

200 응답은 target·diff·rate metadata를 반환한다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

429만 Retry-After와 최대 횟수 안에서 재시도한다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

PR 정보를 안전하게 수집하는 ingestion adapter

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — github_read_result.json을 운영 검토 자료로 사용

포트폴리오 적용

PR 정보를 안전하게 수집하는 ingestion adapter

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — github_read_result.json과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 200 응답은 target-diff-rate metadata를 반환한다.로 정의하는가

복구

재인증·권한 확인·target 확인·diff 갱신·대기 경로를 error code별로 나눈다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

github_read_result.json에서 어떤 status·error_code를 보는가

3차시 완료 확인

- 01 — 설명: REST 상태 코드와 복구이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: `github_read_result.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: LangGraph State와 분기

DAY 4 · 4차시

12:00-12:50 (쉬는 시간 13:40-14:00)

LangGraph State와 분기

State · Node · compiled_review_graph

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/langgraph_lab.py
src/meeting_agent_workflow.py

확인할 결과

compiled_review_graph

4차시 입력과 결과

입력	——	github_read_result.json
이번 차시	——	LangGraph State와 분기
결과	——	compiled_review_graph
다음 연결	——	Checkpoint와 Idempotency

문제와 필요성

모든 원문과 client 객체를 state에 넣으면 checkpoint와 재시작이 불안정해진다.

PR review의 node·edge·conditional edge를 조립한다.

이번 차시의 확인 결과: `compiled_review_graph`

차시 목표

- 01 — PR review의 node·edge·conditional edge를 조립한다.
- 02 — state에 token·client·raw audio를 넣어 checkpoint에 남긴다. 왜 위험한지 설명한다.
- 03 — compiled_review_graph에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
State	node 사이에 전달할 명시 데이터
Node	한 책임을 실행하는 함수
Edge	다음 실행 순서
Conditional edge	상태에 따른 분기

입력·처리·결과·검증

INPUT

src/langgraph_lab.py

OUTPUT

compiled_review_graph

PROCESS

validate target → load diff → review

VERIFY

approve·edit·reject가 서로 다른 terminal state로 간다.
validation error는 publish node에 도달하지 않는다.

5단계 실행 흐름



마지막 판단은 `compiled_review_graph`에서 사람이 확인합니다.

성공·실패 계약

정상	approve·edit·reject가 서로 다른 terminal state로 간다.
경계	validation error는 publish node에 도달하지 않는다.
외부 쓰기	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	status·error_code·입력 식별자·compiled_review_graph

저장소 파일 지도

입력·fixture

src/langgraph_lab.py

구현 코드

src/meeting_agent_workflow.py

검증·test

src/course_services/github_service.py

따라하기 notebook

materials/day4/day4_service_lab.ipynb

개념과 코드 연결

모든 원문과 client 객체를 state에 넣으면 checkpoint와 재시작이 불안정해진다.

ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.

이 차시에서
버릴 오해

state에 token·client·raw
audio를 넣어 checkpoint
에 남긴다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
State	node 사이에 전달할 명시 데이터	결정론 test로 먼저 확인
Node	한 책임을 실행하는 함수	adapter 경계를 확인
Edge	다음 실행 순서	비용·지연·권한을 확인
Conditional edge	상태에 따른 분기	사람 판단과 운영 기록을 확인

대표 실패

실패

state에 token·client·raw audio를
넣어 checkpoint에 남긴다.

사용자 영향

모든 원문과 client 객체를 state에 넣으면
checkpoint와 재시작이 불안정해진다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: validation error는 publish node에 도달하지 않는다.
- 02 — 중단: state에 token·client·raw audio를 넣어 checkpoint에 남긴다.
- 03 — 복구: ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.
- 04 — 증거: compiled_review_graph과 test 결과를 함께 남긴다.

Codex 검증 루프

기존 service 함수를 node로
감싸되 contract를 바꾸지 않게
한다.

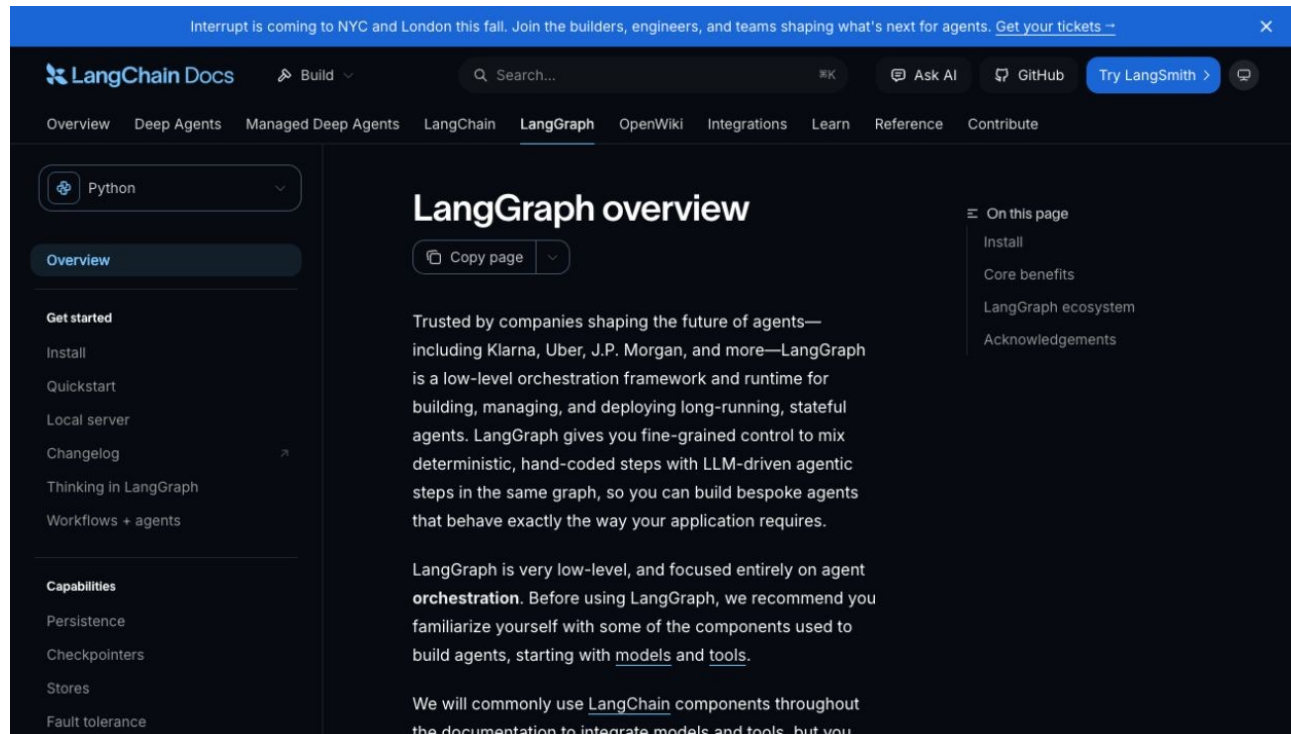
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex 작업 명세

목표

기존 service 함수를 node로 감싸되 contract를 바꾸지 않게 한다.

허용 경로

src/meeting_agent_workflow.py
src/course_services/github_service.py

완료 조건

approve·edit·reject가 서로 다른 terminal state로 간다.
validation error는 publish node에 도달하지 않는다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | approve·edit·reject가 서로 다른 terminal state로 간다. |
| 03 | Boundary test | —— | validation error는 publish node에 도달하지 않는다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사 정상 경로 시연

- 01 — 열기: `src/langgraph_lab.py`
- 02 — 구현 확인: `src/meeting_agent_workflow.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: `approve·edit·reject`가 서로 다른 terminal state로 간다.

실행 명령

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

compiled_review_graph

사람이 확인할 세 줄

1. approve·edit·reject가 서로 다른 terminal state로 간다.
2. validation error는 publish node에 도달하지 않는다.
3. external_write=false

강사 실패·복구 시연

재현

state에 token·client·raw audio를 넣어
checkpoint에 남긴다.

복구

ID·status·검증 결과만 state에 두고
외부 자원은 adapter 경계에서
주입한다.

확인: validation error는 publish node에 도달하지 않는다.

Codex 기반 소프트웨어 제작

열 파일	<code>src/meeting_agent_workflow.py</code> <code>src/course_services/github_service.py</code>
작업 범위	Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	approve·edit·reject가 서로 다른 terminal state로 간다. compiled_review_graph 저장

STEP 1 · 작업 명세 입력

기존 service 함수를 node로 감싸되 contract를 바꾸지 않게 한다.
허용 파일: src/meeting_agent_workflow.py ·
src/course_services/github_service.py

결과 파일: compiled_review_graph

STEP 2 · Patch 생성과 Focused Test

```
python -m src.langgraph_lab --decision all
```

정상 Case: approve·edit·reject가 서로 다른 terminal state로 간다.

결과 파일: compiled_review_graph

STEP 3 · Diff 리뷰와 경계 Case

validation error는 publish node에 도달하지 않는다.
복구: ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서
주입한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 Case Test

NORMAL

approve·edit·reject가 서로 다른 terminal state로 간다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

validation error는 publish node에 도달하지 않는다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

회의 Agent와 PR Agent가 같은 승인 패턴을 재사용

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — compiled_review_graph을 운영 검토 자료로 사용

포트폴리오 적용

회의 Agent와 PR Agent가 같은 승인 패턴을 재사용

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — compiled_review_graph과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 approve·edit·reject가 서로 다른 terminal state로 간다.로 정의하는가

복구

ID·status·검증 결과만 state에 두고 외부 자원은 adapter 경계에서 주입한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

compiled_review_graph에서 어떤 status·error_code를 보는가

4차시 완료 확인

- 01 — 설명: LangGraph State와 분기이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: `compiled_review_graph`과 정상·실패 test 결과가 남아 있다.

다음 차시: Checkpoint와 Idempotency

DAY 4 · 5차시

12:50-13:40 (쉬는 시간 13:40-14:00)

Checkpoint와 Idempotency

Checkpoint · Thread ID · checkpoint_and_audit.json

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/github_service.py
src/langgraph_lab.py

확인할 결과

checkpoint_and_audit.json

5차시 입력과 결과

입력	——	compiled_review_graph
이번 차시	——	Checkpoint와 Idempotency
결과	——	checkpoint_and_audit.json
다음 연결	——	Human Approval과 Interrupt

문제와 필요성

interrupt 전후 node가 다시 실행돼도 외부 쓰기는 한 번이어야 한다.

retry·checkpoint·request ID·idempotency를 연결한다.

이번 차시의 확인 결과: checkpoint_and_audit.json

차시 목표

- 01 — retry·checkpoint·request ID·idempotency를 연결한다.
- 02 — timeout 뒤 동일 comment를 다시 게시한다. 왜 위험한지 설명한다.
- 03 — checkpoint_and_audit.json에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
Checkpoint	중단 시 state 저장
Thread ID	같은 실행을 이어 가는 식별자
Idempotency	같은 요청을 반복해도 결과가 한 번인 성질
Backoff	재시도 간격을 점차 늘림

입력·처리·결과·검증

INPUT

src/course_services/github_service.py

OUTPUT

checkpoint_and_audit.json

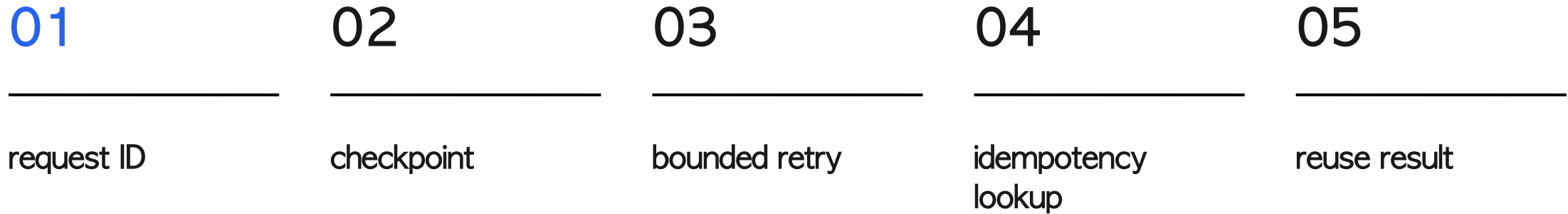
PROCESS

request ID → checkpoint → bounded retry

VERIFY

첫 실행은 reused=false, 두 번째는 true다.
body나 head SHA가 달라지면 새 승인 대상으로 본다.

5단계 실행 흐름



마지막 판단은 `checkpoint_and_audit.json`에서 사람이 확인합니다.

성공·실패 계약

정상	——	첫 실행은 reused=false, 두 번째는 true다.
경계	——	body나 head SHA가 달라지면 새 승인 대상으로 본다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·checkpoint_and_audit.json

저장소 파일 지도

입력·fixture

src/course_services/github_service.py

구현 코드

src/langgraph_lab.py

검증·test

tests/test_course_services.py

따라하기 notebook

materials/day4/day4_service_lab.ipynb

개념과 코드 연결

interrupt 전후 node가 다시 실행돼도 외부 쓰기는 한 번이어야 한다.

target:SHA:body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.

이 차시에서
버릴 오해

timeout 뒤 동일
comment를 다시
게시한다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
Checkpoint	중단 시 state 저장	결정론 test로 먼저 확인
Thread ID	같은 실행을 이어 가는 식별자	adapter 경계를 확인
Idempotency	같은 요청을 반복해도 결과가 한 번인 성질	비용·지연·권한을 확인
Backoff	재시도 간격을 점차 늘림	사람 판단과 운영 기록을 확인

대표 실패

실패

timeout 뒤 동일 comment를 다시
게시한다.

사용자 영향

interrupt 전후 node가 다시 실행되도 외부
쓰기는 한 번이어야 한다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: body나 head SHA가 달라지면 새 승인 대상으로 본다.
- 02 — 중단: timeout 뒤 동일 comment를 다시 게시한다.
- 03 — 복구: target·SHA·body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.
- 04 — 증거: checkpoint_and_audit.json과 test 결과를 함께 남긴다.

Codex 검증 루프

같은 plan을 두 번 실행해
publisher call이 1회인지 test한
다.

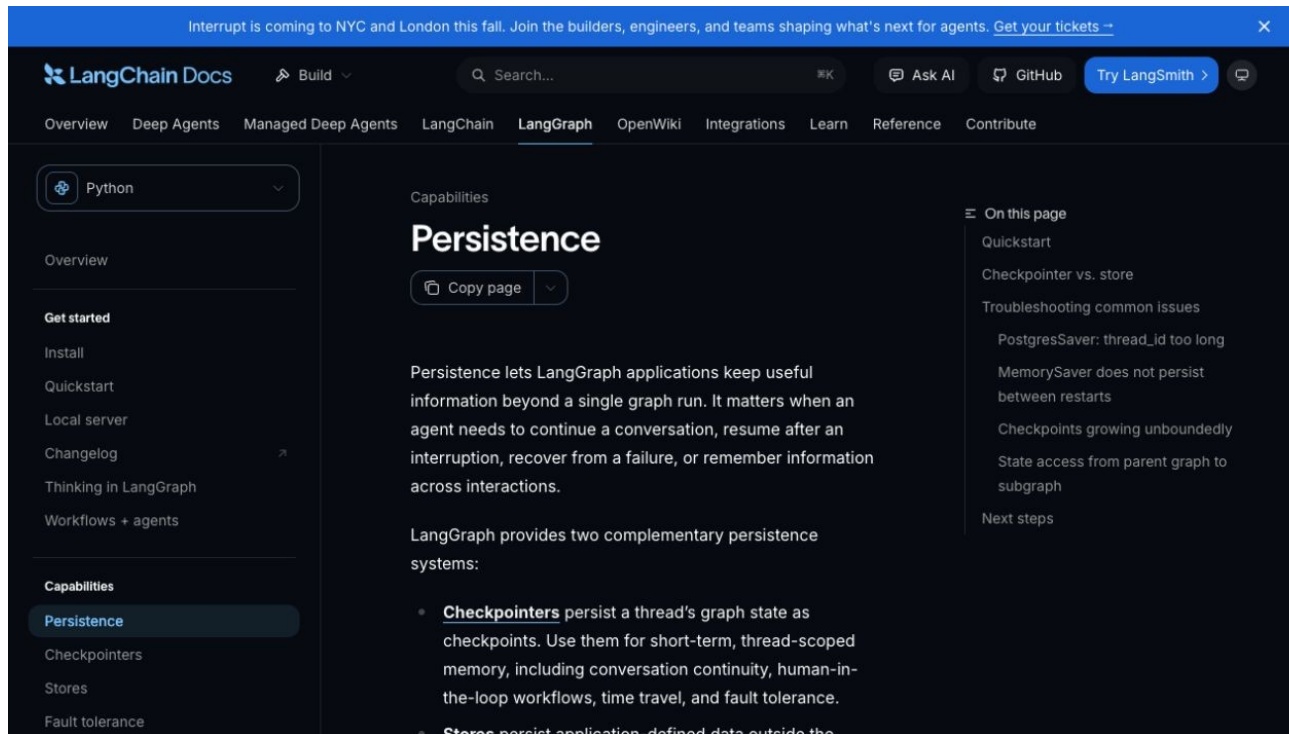
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex 작업 명세

목표

같은 plan을 두 번 실행해 publisher call이 1회인지 test한다.

허용 경로

src/langgraph_lab.py
tests/test_course_services.py

완료 조건

첫 실행은 reused=false, 두 번째는 true다.
body나 head SHA가 달라지면 새 승인 대상으로 본다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- 01 **Scope** — 허용 경로 밖 파일이 바뀌지 않았는가
- 02 **Focused test** — 첫 실행은 reused=false, 두 번째는 true다.
- 03 **Boundary test** — body나 head SHA가 달라지면 새 승인 대상으로 본다.
- 04 **Human merge** — Codex 리뷰와 test 통과는 자동 승인이 아니다

강사 정상 경로 시연

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `src/langgraph_lab.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k idempotent`
- 04 — 성공 신호: 첫 실행은 `reused=false`, 두 번째는 `true`다.

실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k idempotent
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

checkpoint_and_audit.json

사람이 확인할 세 줄

1. 첫 실행은 reused=false, 두 번째는 true다.
2. body나 head SHA가 달라지면 새 승인 대상으로 본다.
3. external_write=false

강사 실패·복구 시연

재현

timeout 뒤 동일 comment를 다시
게시한다.

복구

target·SHA·body hash를 idempotency
key로 저장하고 기존 remote result를
재사용한다.

확인: body나 head SHA가 달라지면 새 승인 대상으로 본다.

Codex 기반 소프트웨어 제작

열 파일 `src/langgraph_lab.py`
 `tests/test_course_services.py`

작업 범위 Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 첫 실행은 `reused=false`, 두 번째는 `true`다.
 `checkpoint_and_audit.json` 저장

STEP 1 · 작업 명세 입력

같은 plan을 두 번 실행해 publisher call이 1회인지 test한다.
허용 파일: `src/langgraph_lab.py · tests/test_course_services.py`

결과 파일: `checkpoint_and_audit.json`

STEP 2 · Patch 생성과 Focused Test

```
python -m pytest -q tests/test_course_services.py -k idempotent
```

정상 Case: 첫 실행은 reused=false, 두 번째는 true다.

결과 파일: checkpoint_and_audit.json

STEP 3 · Diff 리뷰와 경계 Case

body나 head SHA가 달라지면 새 승인 대상으로 본다.

복구: target·SHA·body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 Case Test

NORMAL

첫 실행은 reused=false, 두 번째는 true다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

body나 head SHA가 달라지면 새 승인 대상으로 본다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

네트워크가 불안한 CI bot의 중복 comment 방지

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — checkpoint_and_audit.json을 운영 검토 자료로 사용

포트폴리오 적용

네트워크가 불안한 CI bot의 중복 comment 방지

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — checkpoint_and_audit.json과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 첫 실행은 reused=false, 두 번째는 true다.로 정의하는가

복구

target-SHA-body hash를 idempotency key로 저장하고 기존 remote result를 재사용한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

checkpoint_and_audit.json에서 어떤 status:error_code를 보는가

5차시 완료 확인

- 01 — 설명: Checkpoint와 Idempotency이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k idempotent`
- 03 — 증거: `checkpoint_and_audit.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: Human Approval과 Interrupt

DAY 4 · 6차시

15:00-15:50 (쉬는 시간 17:30-18:00)

Human Approval과 Interrupt

Interrupt · Approve · review_decision.json

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/langgraph_lab.py
src/meeting_agent_workflow.py

확인할 결과

review_decision.json

6차시 입력과 결과

입력	————	checkpoint_and_audit.json
이번 차시	————	Human Approval과 Interrupt
결과	————	review_decision.json
다음 연결	————	Dry-run Comment Payload

문제와 필요성

초안만 보여주고 대상과 근거를 숨기면 사람은 사실상 승인할 수 없다.

approve·edit·reject interrupt payload를 설계한다.

이번 차시의 확인 결과: review_decision.json

차시 목표

- 01 — approve·edit·reject interrupt payload를 설계한다.
- 02 — interrupt를 일반 Exception으로 잡아 자동 진행한다. 왜 위험한지 설명한다.
- 03 — review_decision.json에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
Interrupt	workflow를 멈추고 입력을 기다림
Approve	초안을 그대로 허용
Edit	사람이 수정 후 허용
Reject	외부 쓰기 없이 종료

입력·처리·결과·검증

INPUT

src/langgraph_lab.py

OUTPUT

review_decision.json

PROCESS

초안 생성 → target 표시 → finding 근거

VERIFY

approve가 READY_FOR_EXPORT로 간다.
reject는 external_write=false와 REJECTED로 끝난다.

5단계 실행 흐름

01

초안 생성

02

target 표시

03

finding 근거

04

사람 결정

05

같은 thread resume

마지막 판단은 review_decision.json에서 사람이 확인합니다.

성공·실패 계약

정상	approve가 READY_FOR_EXPORT로 간다.
경계	reject는 external_write=false와 REJECTED로 끝난다.
외부 쓰기	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	status·error_code·입력 식별자·review_decision.json

저장소 파일 지도

입력·fixture

src/langgraph_lab.py

구현 코드

src/meeting_agent_workflow.py

검증·test

web-demo/app.js

따라하기 notebook

materials/day4/day4_service_lab.ipynb

개념과 코드 연결

초안만 보여주고 대상과 근거를 숨기면 사람은 사실상 승인할 수 없다.

interrupt를 workflow event로 두고 같은 thread_id와 JSON decision으로 resume한다.

이 차시에서
버릴 오해

interrupt를 일반
Exception으로 잡아
자동 진행한다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
Interrupt	workflow를 멈추고 입력을 기다림	결정론 test로 먼저 확인
Approve	초안을 그대로 허용	adapter 경계를 확인
Edit	사람이 수정 후 허용	비용·지연·권한을 확인
Reject	외부 쓰기 없이 종료	사람 판단과 운영 기록을 확인

대표 실패

실패

interrupt를 일반 Exception으로 잡아
자동 진행한다.

사용자 영향

초안만 보여주고 대상과 근거를 숨기면
사람은 사실상 승인할 수 없다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: reject는 external_write=false와 REJECTED로 끝난다.
- 02 — 중단: interrupt를 일반 Exception으로 잡아 자동 진행한다.
- 03 — 복구: interrupt를 workflow event로 두고 같은 thread_id와 JSON decision으로 resume한다.
- 04 — 증거: review_decision.json과 test 결과를 함께 남긴다.

Codex 검증 루프

승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.

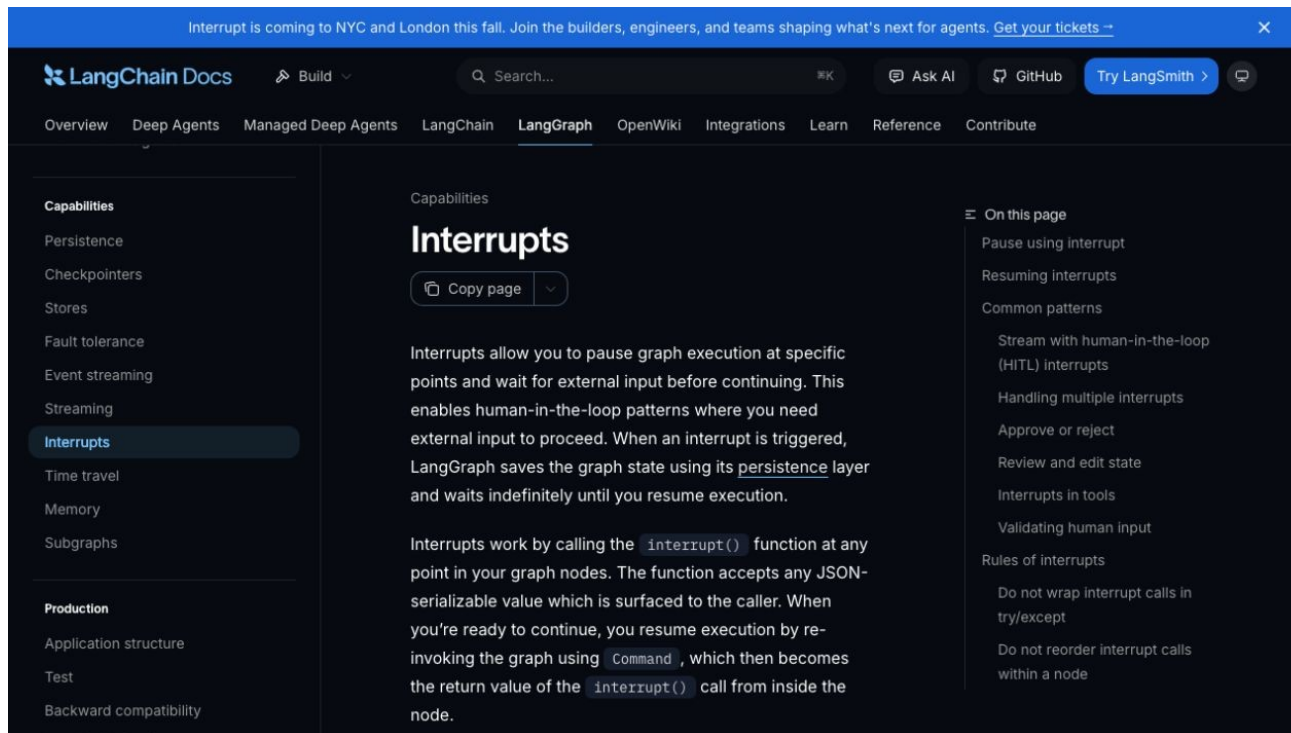
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex 작업 명세

목표

승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.

허용 경로

src/meeting_agent_workflow.py
web-demo/app.js

완료 조건

approve가 READY_FOR_EXPORT로 간다.
reject는 external_write=false와 REJECTED로 끝난다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | approve가 READY_FOR_EXPORT로 간다. |
| 03 | Boundary test | —— | reject는 external_write=false와 REJECTED로 끝난다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사 정상 경로 시연

- 01 — 열기: `src/langgraph_lab.py`
- 02 — 구현 확인: `src/meeting_agent_workflow.py`
- 03 — 실행: `python -m src.langgraph_lab --decision all`
- 04 — 성공 신호: approve가 READY_FOR_EXPORT로 간다.

실행 명령

```
$ python -m src.langgraph_lab --decision all
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

review_decision.json

사람이 확인할 세 줄

1. approve가 READY_FOR_EXPORT로 간다.
2. reject는 external_write=false와 REJECTED로 끝난다.
3. external_write=false

강사 실패·복구 시연

재현

interrupt를 일반 Exception으로 잡아
자동 진행한다.

복구

interrupt를 workflow event로 두고
같은 thread_id와 JSON decision으로
resume한다.

확인: reject는 external_write=false와 REJECTED로 끝난다.

Codex 기반 소프트웨어 제작

열 파일 `src/meeting_agent_workflow.py`
 `web-demo/app.js`

작업 범위 Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 approve가 READY_FOR_EXPORT로 간다.
 `review_decision.json` 저장

STEP 1 · 작업 명세 입력

승인 node 전에 외부 쓰기 호출이 존재하지 않는지 diff review한다.
허용 파일: src/meeting_agent_workflow.py · web-demo/app.js

결과 파일: review_decision.json

STEP 2 · Patch 생성과 Focused Test

```
python -m src.langgraph_lab --decision all
```

정상 Case: approve가 READY_FOR_EXPORT로 간다.

결과 파일: review_decision.json

STEP 3 · Diff 리뷰와 경계 Case

reject는 `external_write=false`와 `REJECTED`로 끝난다.

복구: `interrupt`를 `workflow event`로 두고 같은 `thread_id`와 `JSON decision`으로 `resume`한다.

결과에 `error_code`와 `external_write=false`가 보이면 완료

정상 Case Test

NORMAL

approve가 READY_FOR_EXPORT로 간다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

reject는 `external_write=false`와 REJECTED로 끝난다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

PR comment·메일·ticket 생성 전에 동일 승인 UI 사용

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_decision.json을 운영 검토 자료로 사용

포트폴리오 적용

PR comment·메일·ticket 생성 전에 동일 승인 UI 사용

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review_decision.json과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 approve가 READY_FOR_EXPORT로 간다.로 정의하는가

복구

interrupt를 workflow event로 두고 같은 thread_id와 JSON decision으로 resume한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

review_decision.json에서 어떤 status·error_code를 보는가

6차시 완료 확인

- 01 — 설명: Human Approval과 Interrupt이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m src.langgraph_lab --decision all`
- 03 — 증거: `review_decision.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: Dry-run Comment Payload

DAY 4 · 7차시

15:50-16:40 (쉬는 시간 17:30-18:00)

Dry-run Comment Payload

Dry-run · Payload · review_comment_plan.json

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

src/course_services/github_service.py
src/course_services/contracts.py

확인할 결과

review_comment_plan.json

7차시 입력과 결과

입력	————	review_decision.json
이번 차시	————	Dry-run Comment Payload
결과	————	review_comment_plan.json
다음 연결	————	Sandbox GitHub 게시

문제와 필요성

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

PR comment body·target·idempotency key를 local JSON으로 만든다.

이번 차시의 확인 결과: review_comment_plan.json

차시 목표

- 01 — PR comment body·target·idempotency key를 local JSON으로 만든다.
- 02 — dry_run=true인데 publisher가 호출된다. 왜 위험한지 설명한다.
- 03 — review_comment_plan.json에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
Dry-run	외부 쓰기 없이 동일 payload 생성
Payload	API에 보낼 구조화 데이터
Audit	누가 무엇을 승인했는지 기록
External write	외부 시스템 상태를 바꾸는 호출

입력·처리·결과·검증

INPUT

src/course_services/github_service.py

OUTPUT

review_comment_plan.json

PROCESS

report validate → comment render → target bind

VERIFY

payload에 repo·PR·SHA·body·key가 있다.
dry-run plan은 approval=true여도 publish되지 않는다.

5단계 실행 흐름

01

report validate

02

comment render

03

target bind

04

hash key

05

READY_FOR_APPR
OVAL

마지막 판단은 review_comment_plan.json에서 사람이 확인합니다.

성공·실패 계약

정상	——	payload에 repo·PR·SHA·body·key가 있다.
경계	——	dry-run plan은 approval=true여도 publish되지 않는다.
외부 쓰기	——	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	——	status·error_code·입력 식별자·review_comment_plan.json

저장소 파일 지도

입력·fixture

src/course_services/github_service.py

구현 코드

src/course_services/contracts.py

검증·test

tests/test_course_services.py

따라하기 notebook

materials/day4/day4_service_lab.ipynb

개념과 코드 연결

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

DRY_RUN_CANNOT_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.

이 차시에서
버릴 오해

`dry_run=true`인데
publisher가 호출된다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
Dry-run	외부 쓰기 없이 동일 payload 생성	결정론 test로 먼저 확인
Payload	API에 보낼 구조화 데이터	adapter 경계를 확인
Audit	누가 무엇을 승인했는지 기록	비용·지연·권한을 확인
External write	외부 시스템 상태를 바꾸는 호출	사람 판단과 운영 기록을 확인

대표 실패

실패

`dry_run=true`인데 publisher가 호출된다.

사용자 영향

대충 만든 preview는 실제 API payload와 달라 승인 증거가 되지 못한다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: dry-run plan은 approval=true여도 publish되지 않는다.
- 02 — 중단: dry_run=true인데 publisher가 호출된다.
- 03 — 복구: DRY_RUN_CANNOT_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.
- 04 — 증거: review_comment_plan.json과 test 결과를 함께 남긴다.

Codex 검증 루프

dry-run 경계를 건드리는
변경에는 boundary test 없이는
완료하지 않는다.

저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge

WORKFLOW LAB

PipelineReviewResult

03

COMMAND RESUME

같은 thread의 최종 상태를 확인합니다.

TERMINAL STATUS

REJECTED

외부로 내보내지 않고 검토 상태로 종료합니다.

RELEASE GATE

HOLD

Schema·근거·사람 결정·외부 쓰기 금지를 검사합니다.

SIDE EFFECT

LOCAL JSON ONLY

메일·결제·고객 데이터 변경을 수행하지 않습니다.

AUDIT EVENTS

1. validate

REVIEW_REQUIRED

2. review

reject

3. rejected

OUTPUT

내 결정이 담긴 JSON

브라우저 Demo 결과를 파일로 저장해 Git commit 또는 포트폴리오 증거로 사용할 수 있습니다.

Codex 작업 명세

목표

dry-run 경계를 건드리는 변경에는 boundary test 없이는 완료하지 않는다.

허용 경로

src/course_services/contracts.py
tests/test_course_services.py

완료 조건

payload에 repo·PR·SHA·body·key가 있다.
dry-run plan은 approval=true여도 publish되지 않는다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- | | | | |
|----|---------------|----|--|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | payload에 repo·PR·SHA·body·key가 있다. |
| 03 | Boundary test | —— | dry-run plan은 approval=true여도 publish되지 않는다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사 정상 경로 시연

- 01 — 열기: `src/course_services/github_service.py`
- 02 — 구현 확인: `src/course_services/contracts.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py -k dry_run`
- 04 — 성공 신호: payload에 `repo·PR·SHA·body·key`가 있다.

실행 명령

```
$ python -m pytest -q tests/test_course_services.py -k dry_run
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

review_comment_plan.json

사람이 확인할 세 줄

1. payload에 repo·PR·SHA·body·key가 있다.
2. dry-run plan은 approval=true여도 publish 되지 않는다.
3. external_write=false

강사 실패·복구 시연

재현

`dry_run=true`인데 publisher가 호출된다.

복구

`DRY_RUN_CANNOT_PUBLISH`를 contract로 고정하고 fake publisher call 수를 assert한다.

확인: dry-run plan은 `approval=true`여도 publish되지 않는다.

Codex 기반 소프트웨어 제작

열 파일 `src/course_services/contracts.py`
 `tests/test_course_services.py`

작업 범위 Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토

완료 기준 payload에 repo·PR·SHA·body·key가 있다.
 review_comment_plan.json 저장

STEP 1 · 작업 명세 입력

dry-run 경계를 건드리는 변경에는 boundary test 없이는 완료하지 않는다.
허용 파일: `src/course_services/contracts.py` · `tests/test_course_services.py`

결과 파일: `review_comment_plan.json`

STEP 2 · Patch 생성과 Focused Test

```
python -m pytest -q tests/test_course_services.py -k dry_run
```

정상 Case: payload에 repo·PR·SHA·body·key가 있다.

결과 파일: review_comment_plan.json

STEP 3 · Diff 리뷰와 경계 Case

dry-run plan은 approval=true여도 publish되지 않는다.

복구: DRY_RUN_CANNOT_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 Case Test

NORMAL

payload에 repo·PR·SHA·body·key가 있다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

dry-run plan은 approval=true여도 publish되지 않는다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

운영자가 Slack/PR 화면에서 실제 게시 내용을 사전 검토

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — review_comment_plan.json을 운영 검토 자료로 사용

포트폴리오 적용

운영자가 Slack/PR 화면에서 실제 게시 내용을 사전 검토

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — review_comment_plan.json과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 payload에 repo·PR·SHA·body·key가 있다.로 정의하는가

복구

DRY_RUN_CANNOT_PUBLISH를 contract로 고정하고 fake publisher call 수를 assert한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

review_comment_plan.json에서 어떤 status·error_code를 보는가

7차시 완료 확인

- 01 — 설명: Dry-run Comment Payload이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py -k dry_run`
- 03 — 증거: `review_comment_plan.json`과 정상·실패 test 결과가 남아 있다.

다음 차시: Sandbox GitHub 게시

DAY 4 · 8차시

16:40-17:30 (쉬는 시간 17:30-18:00)

Sandbox GitHub 게시

Sandbox repo · Publisher · day4_audit_record.json

진행

강의 12분 · 강사 시연 10분 · Codex
소프트웨어 제작 23분 · 실행 확인 5분

사용 파일

materials/day4/day4_service_lab.ipynb
src/course_services/github_service.py

확인할 결과

day4_audit_record.json

8차시 입력과 결과

입력 ————— review_comment_plan.json

이번 차시 ————— Sandbox GitHub 게시

결과 ————— day4_audit_record.json

다음 연결 ————— 17:30-18:00 쉬는 시간

문제와 필요성

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

선택적으로 승인된 comment 한 건을 게시하고 audit ID를 남긴다.

이번 차시의 확인 결과: day4_audit_record.json

차시 목표

- 01 — 선택적으로 승인된 comment 한 건을 게시하고 audit ID를 남긴다.
- 02 — 교육 코드에 live publisher와 token scope를 기본 포함한다. 왜 위험한지 설명한다.
- 03 — day4_audit_record.json에서 정상과 실패 상태를 직접 확인한다.

핵심 용어 4가지

용어	이 수업에서 쓰는 뜻
Sandbox repo	실습 전용 본인 저장소
Publisher	외부 API write adapter
Remote ID	게시 결과 식별자
Rollback	comment 삭제 또는 commit revert

입력·처리·결과·검증

INPUT

materials/day4/day4_service_lab.ipynb

OUTPUT

day4_audit_record.json

PROCESS

sandbox 확인 → dry-run 검토 → 사람 승인

VERIFY

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
publisher 미설정·승인 없음·dry-run은 모두 BLOCKED
다.

5단계 실행 흐름

01

sandbox 확인

02

dry-run 검토

03

사람 승인

04

1회 POST

05

response ID 기록

마지막 판단은 day4_audit_record.json에서 사람이 확인합니다.

성공·실패 계약

정상	————	fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
경계	————	publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
외부 쓰기	————	기본값 false · dry-run과 사람 승인 뒤에만 별도 adapter 호출
기록	————	status·error_code·입력 식별자·day4_audit_record.json

저장소 파일 지도

입력·fixture

materials/day4/day4_service_lab.ipynb

구현 코드

src/course_services/github_service.py

검증·test

.env.sample

따라하기 notebook

AGENTS.md

개념과 코드 연결

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

이 차시에서
버릴 오해

교육 코드에 live
publisher와 token scope
를 기본 포함한다.

역할과 선택 기준

구분	역할	이번 차시의 판단 기준
Sandbox repo	실습 전용 본인 저장소	결정론 test로 먼저 확인
Publisher	외부 API write adapter	adapter 경계를 확인
Remote ID	게시 결과 식별자	비용·지연·권한을 확인
Rollback	comment 삭제 또는 commit revert	사람 판단과 운영 기록을 확인

대표 실패

실패

교육 코드에 live publisher와 token scope를 기본 포함한다.

사용자 영향

교육 실습이 타인 저장소나 업무 repo를 오염시키면 안 된다.

실패가 재현돼야 복구 코드와 회귀 test를 만들 수 있습니다.

복구 경로

- 01 — 탐지: publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
- 02 — 중단: 교육 코드에 live publisher와 token scope를 기본 포함한다.
- 03 — 복구: repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.
- 04 — 증거: day4_audit_record.json과 test 결과를 함께 남긴다.

Codex 검증 루프

PR 생성·리뷰·test 수정은 돕되
merge와 외부 게시 결정은
사람에게 남긴다.

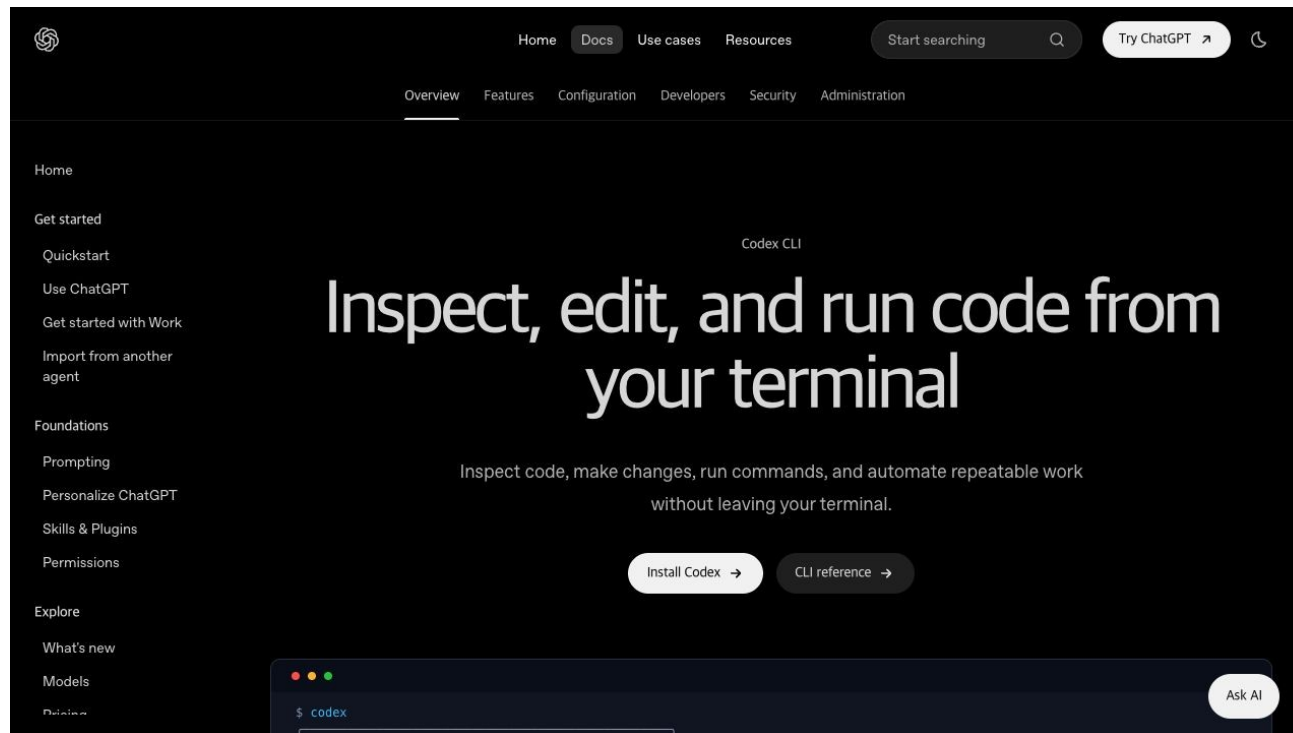
저장소 이해

→ 허용 범위 변경

→ focused test

→ diff 리뷰

→ 사람 merge



Codex 작업 명세

목표

PR 생성·리뷰·test 수정은 돕되 merge와 외부 게시 결정은 사람에게 남긴다.

허용 경로

src/course_services/github_service.py
.env.sample

완료 조건

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

금지

secret 출력 · workspace 밖 변경 · test 약화 · 사람 승인 없는 외부 쓰기

Patch 검증과 사람 Merge

- | | | | |
|----|---------------|----|---|
| 01 | Scope | —— | 허용 경로 밖 파일이 바뀌지 않았는가 |
| 02 | Focused test | —— | fake publisher가 한 번 호출되고 remote ID가 audit에 남는다. |
| 03 | Boundary test | —— | publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다. |
| 04 | Human merge | —— | Codex 리뷰와 test 통과는 자동 승인이 아니다 |

강사 정상 경로 시연

- 01 — 열기: `materials/day4/day4_service_lab.ipynb`
- 02 — 구현 확인: `src/course_services/github_service.py`
- 03 — 실행: `python -m pytest -q tests/test_course_services.py`
- 04 — 성공 신호: fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

실행 명령

```
$ python -m pytest -q tests/test_course_services.py
```

실행 전 확인

현재 경로가 repository root인지 · Python 3.12 환경인지 · .env가 출력되지 않는지

완료 기준

STATUS

SUCCESS 또는 EXPECTED_FAILURE

ARTIFACT

day4_audit_record.json

사람이 확인할 세 줄

1. fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.
2. publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.
3. external_write=false

강사 실패·복구 시연

재현

교육 코드에 live publisher와 token scope를 기본 포함한다.

복구

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

확인: publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

Codex 기반 소프트웨어 제작

열 파일	<code>src/course_services/github_service.py</code> <code>.env.sample</code>
작업 범위	Codex 작업 명세 → 최소 Patch → Focused Test → Diff 검토
완료 기준	fake publisher가 한 번 호출되고 remote ID가 audit에 남는다. day4_audit_record.json 저장

STEP 1 · 작업 명세 입력

PR 생성·리뷰·test 수정은 돕되 merge와 외부 게시 결정은 사람에게 남긴다.
허용 파일: `src/course_services/github_service.py` · `.env.sample`

결과 파일: `day4_audit_record.json`

STEP 2 · Patch 생성과 Focused Test

```
python -m pytest -q tests/test_course_services.py
```

정상 Case: fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

결과 파일: day4_audit_record.json

STEP 3 · Diff 리뷰와 경계 Case

publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

복구: repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch
에서 연결한다.

결과에 error_code와 external_write=false가 보이면 완료

정상 Case Test

NORMAL

fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.

- 01 — 입력 fixture와 실행 command가 기록됐는가
- 02 — 결과 schema가 validate되는가
- 03 — focused test가 실제로 통과했는가

경계 Case Test

BOUNDARY

publisher 미설정·승인 없음·dry-run은 모두 BLOCKED다.

- 01 — raw traceback 대신 stable error_code가 남는가
- 02 — 외부 쓰기·자동 메일이 false인가
- 03 — 실패가 다음 차시에서 재현 가능한가

현업 적용

현업은 GitHub App과 protected branch, 구직자는 sandbox demo recording

- 01 — 실제 업무 데이터 대신 합성·비식별 sample로 먼저 실행
- 02 — 현재 수작업 시간과 오류를 baseline으로 기록
- 03 — 사람 승인 전에는 외부 시스템을 바꾸지 않음
- 04 — day4_audit_record.json을 운영 검토 자료로 사용

포트폴리오 적용

현업은 GitHub App과 protected branch, 구직자는 sandbox demo recording

- 01 — 문제와 사용자 영향을 한 문장으로 설명
- 02 — 정상 demo와 대표 실패 demo를 함께 준비
- 03 — Codex가 만든 diff와 직접 검토한 test 증거를 구분
- 04 — day4_audit_record.json과 README 재현 절차를 제출

서비스 운영 점검

품질

무엇을 fake publisher가 한 번 호출되고 remote ID가 audit에 남는다.로 정의하는가

복구

repo에는 fake publisher만 두고 live adapter는 강사 승인 후 별도 branch에서 연결한다.

안전

어디에서 사람 승인과 external_write=false를 확인하는가

관측

day4_audit_record.json에서 어떤 status_error_code를 보는가

8차시 완료 확인

- 01 — 설명: Sandbox GitHub 게시이 필요한 이유를 한 문장으로 말할 수 있다.
- 02 — 실행: `python -m pytest -q tests/test_course_services.py`
- 03 — 증거: `day4_audit_record.json`과 정상·실패 test 결과가 남아 있다.

17:30-18:00 쉬는 시간